Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance

Stephen M. Blackburn, Kathryn S. McKinley

Szabó Márk

Garbage Collection Background

Why Garbage Collection Matters

Managed languages (Java, C#) allocate many **small, short-lived** objects, so the garbage collector **directly determines** performance.

Applications make conflicting demands:

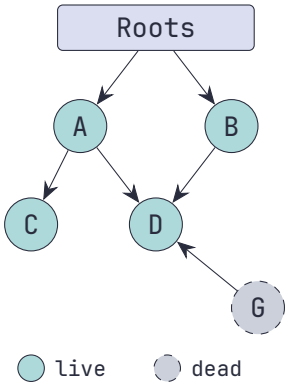
- real-time: **pause-time guarantees**
- embedded: **space efficiency**
- servers: **throughput**

Tracing Garbage Collection

A tracing collector has three jobs:

- **Allocation** of new objects.
- **Identification**: mark every object reachable from the roots.
- **Reclamation** of unmarked objects.

An object is `dead` if we can't reach it from roots.



The Canonical Tracing Collectors

The paper compares immix against the three canonical tracing collectors:

- **Mark-sweep (MS):** non-moving, objects stay where they were allocated.
- **Semi-space (SS):** moving, copies live objects into a second space.
- **Mark-compact (MC):** moving, slides live objects together within the same space.

Building blocks

More sophisticated collectors, such as generational ones, are built by combining these.

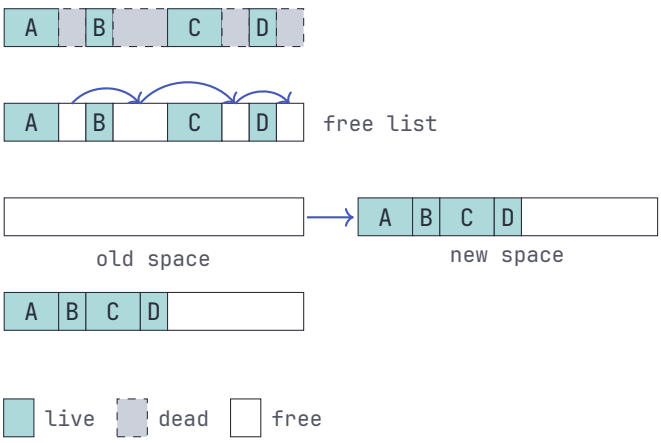
Three Reclamation Strategies

After marking
live and dead objects

Mark-sweep
objects stay in place

Semi-space
evacuate to a new space

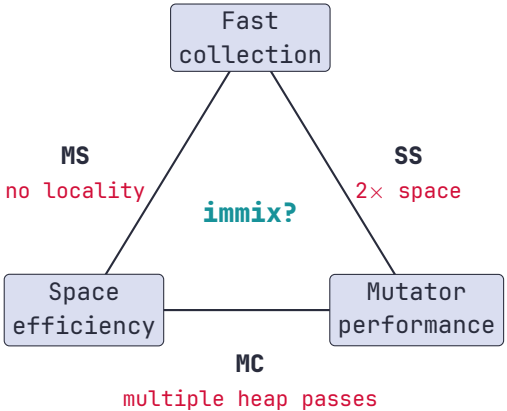
Mark-compact
slide to one end



Three Performance Objectives

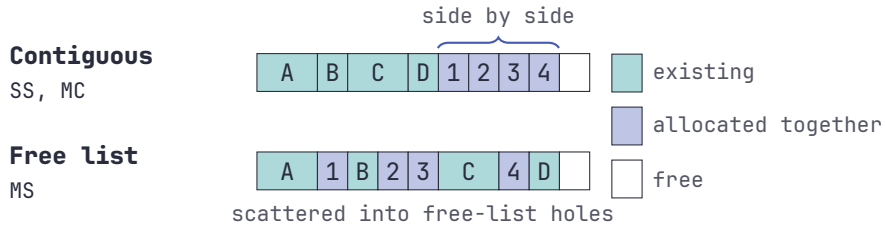
- **Space efficiency:** needs only a small heap.
- **Fast collection:** little time spent collecting.
- **Mutator performance:** the application (*mutator*) runs fast, thanks to locality.

Each canonical collector achieves only two of the three.



Locality and Contiguous Allocation

- **Contiguous allocation** puts objects allocated at the same time next to each other in space: good **mutator locality**.
- **Mark-sweep** is non-moving and allocates from a free list: no locality for objects allocated at the same time.

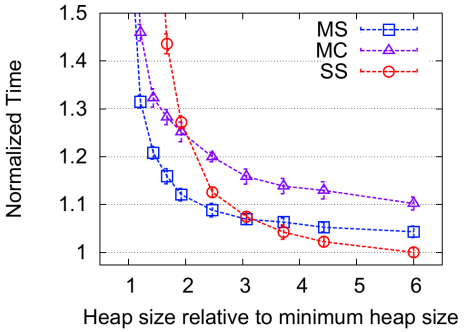


The Space-Time Trade-off

MS, SS and MC from 1× to 6× the minimum heap:

- **Small heap:** GC runs often, fast collection wins (**MS**).
- **Large heap:** GC runs rarely, fast program wins (**SS**).
- **MC:** collecting is too slow.

The crossing lines are the classic **space-time trade-off**.



(a) Total Time

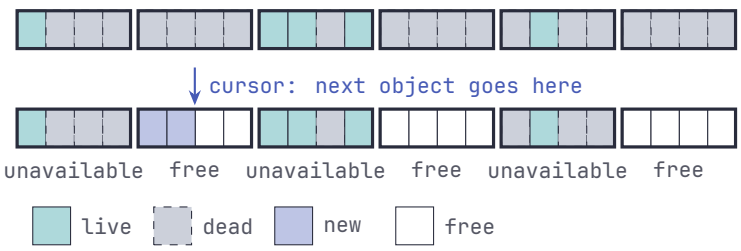
Mark-Region Collection

Mark-Region Collectors

- Memory is divided into **fixed-size regions**. Objects can't span regions.
- **Bump allocation:** each new object goes right after the previous one, by advancing (*bumping*) a pointer. When no free region is left, we mark and sweep again.
- A region holding any live object is *unavailable*, all others are *free*.

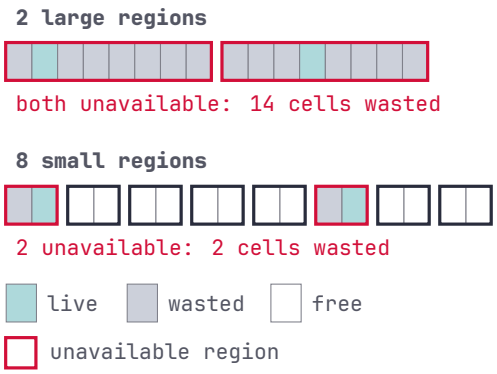
After marking
live and dead objects

After sweep
whole regions reclaimed



Two Open Questions

1. **How big should regions be?**
   - **Large:** one small object wastes a whole region.
   - **Small:** less waste, but slower collection.
2. **How to defragment?**
   - A region with **any** live object stays unavailable.

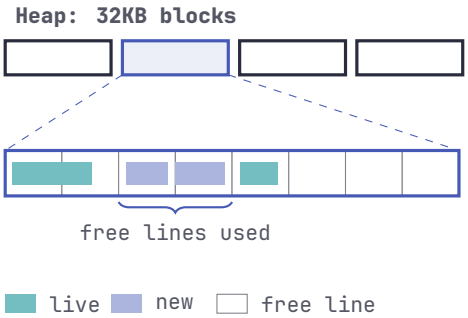


Immix

Blocks and Lines

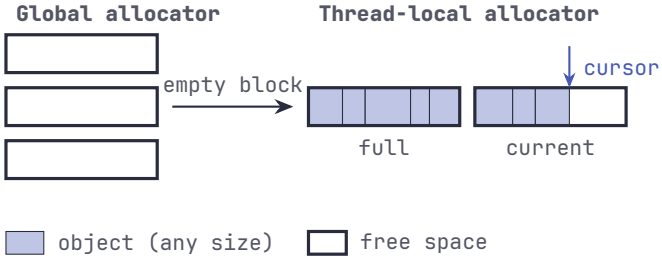
Answers question 1: use big blocks when possible, small lines when necessary.

- 32KB **blocks**, split into 128B **lines**.
- Free block: reclaimed **whole**.
- Partly used block: new objects fill its **free lines**.
- Objects span lines, never blocks.



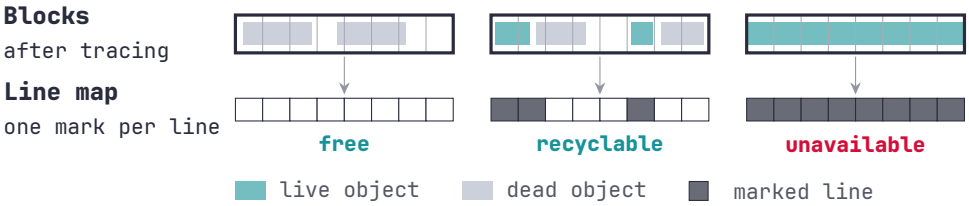
Initial Allocation

- All blocks start **empty**.
- Each thread's allocator takes an empty block from the **global allocator** and bump allocates into it.
- Block full: take another. No empty blocks left: the heap is full, so a collection starts.



Identification and Reclamation

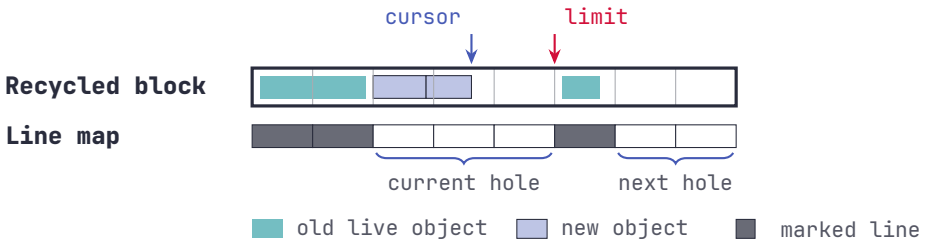
1. **Identification:** trace from the roots and mark each live object **and its lines** in a **line map**.



2. **Reclamation:** read the **line map** and find the entirely free and partially free blocks. Free blocks return to the **global pool** and partially free blocks are **recycled** for the next allocation.

Steady State Allocation

- Allocate into **recycled blocks** first, in address order. Empty blocks come last.
- Scan the line map for a **hole** (one or more free lines in a row) and bump allocate into it.
- **Limit:** the next marked line or the block's end. Past it: find the next hole, or take a new block.



Exact Line Marking

- Every line a live object lies on must be marked, or its space could be reused.
- **Exact marking**, for each live object:
  1. read its type to find its **size**,
  2. loop over its memory, marking **each line**.
- **Mark-sweep** just sets **one bit** in the header compared to line marking which is expensive.

Key observation

Most objects are smaller than a line (128B), so they span **at most two lines**.

Conservative Line Marking

- **Small object** (up to a line): mark only its **start line**. No size lookup, one write.
- Its possible second line: the allocator **skips the first line** of every hole.
- **Medium object** (larger than a line): marked exactly, but rare.
- One **header bit**, set at allocation, says small or medium.



Immix Heap Organization

TODO: Walk through the paper's figure: global free block allocator, recycled blocks, cursor and limit, line mark legend.

Copy from the paper: Figure 2 (immix heap organization)

Recycling and Allocation Policies

TODO: A block with at least $F = 1$ free line is recyclable. Recyclable blocks are used in address order, free blocks last because the large object space also needs them.

Parallelism

TODO: Synchronized global block allocator, unsynchronized thread-local allocators. Racy marking is safe; line marks are bytes to avoid synchronization.

Demand-Driven Overflow Allocation

TODO: Medium objects (larger than a line) that do not fit the current hole go to a separate overflow allocator using empty blocks, so small holes are not skipped.

TikZ diagram: medium object too large for the current hole, redirected to an empty overflow block instead of skipping the hole

Opportunistic Defragmentation

Opportunistic Evacuation

Answers question 2: move objects out of fragmented blocks while marking, only when needed.

TODO: Before collection, choose candidate blocks to evacuate and target space. During marking, move objects off candidates and leave forwarding pointers; stop moving when space runs out. 2.5% of the heap is reserved as evacuation headroom.

TikZ diagram: fragmented candidate block, live objects copied into holes of a target block, forwarding pointers left behind

Candidate Selection

TODO: Defragment on demand. Mark and available histograms indexed by hole count; select the most fragmented blocks that fit the available space.

TikZ diagram: mark and available histograms by hole count, with a cut-off line where required space exceeds available space

Mixing Marking and Evacuation

TODO: Per-object decision during the trace: unmarked and unmovable → mark; unmarked and movable → evacuate and forward; forwarded → update reference. Single pass. Racing threads use compare-and-swap to forward an object; pinned objects are never moved, only marked.

TikZ diagram: flowchart of the per-object decision during tracing

Evaluation

Methodology

TODO: 20 DaCapo, SPECjvm98 and pjbb2000 benchmarks; Jikes RVM with MMTk; Core 2 Duo, AMD Athlon, PowerPC 970; replay compilation; heaps from 1× to 6× the minimum.

Full Heap Performance

TODO: Immix beats all canonical collectors at every heap size on all three architectures, matching SS mutator time and MS collection time.

Copy from the paper: Figure 3 (full heap performance comparison)

Full Heap Results at 2× Minimum Heap

TODO: Per-benchmark slice: immix geomean 0.93 vs MS, worst case 1.01, best 0.82 (jython).

Copy from the paper: Table 1 (full heap performance at 2× minimum heap)

Space Efficiency

TODO: Minimum heap size relative to mark-sweep: immix 86%, mark-compact 83%, semi-space far larger.

Copy from the paper: Figure 5 (minimum heap sizes)

Immix in a Generational Collector

TODO: Semi-space nursery with an immix mature space (G|SS-IX) vs the production collector G|SS-MS; slightly better on average, 5% better on pjbb2000.

TikZ diagram: generational heap: bump-allocated nursery, survivors promoted into the immix mature space

Generational Performance at 1.5× Minimum Heap

TODO: Per-benchmark slice normalized to G|SS-MS: G|SS-IX geomean 0.99, in-place G|IX-IX 1.02, G|MS-MS 1.10.

Copy from the paper: Table 2 (generational performance at 1.5× minimum heap)

In-Place Generational: Sticky Mark Bits

TODO: Mark bits stay set between collections, so a nursery collection only traces newly allocated objects without moving anything. A write barrier remembers modified old objects. Supports pinning, unlike a copying nursery.

TikZ diagram: heap after a full collection with old objects marked; new unmarked objects allocated between them; a nursery collection traces only the unmarked ones

Generational Results

TODO: Copying generational, in-place (sticky mark bits) generational, and all immix variants compared.

Copy from the paper: Figure 6 (generational collector performance)

Selected Benchmarks

TODO: javac and luindex: full heap immix is the best collector. pjbb2000: immix variants beat the production collector.

Copy from the paper: Figure 4 (selected benchmarks)

What Makes Immix Work

TODO: Ablation: block-only marking inflates marked space by 93%, lines without defragmentation by 23%. Defragmentation rarely triggers but frees 95% of candidate space. Overflow allocation handles about 4% of allocation.

Conclusion

Related Work

Mark-region collectors existed, in JRockit and IBM's JVM, but were never described or evaluated in a publication.

How immix differs:

- **Defragmentation:** JRockit and IBM use compaction; immix evacuates opportunistically in a **single pass**.
- **Granularity:** IBM never reuses gaps smaller than 512 bytes; immix reclaims **128-byte lines**.
- **Metadata:** 0.8% of the heap for immix vs 3.1% for IBM.

Closest relative

Metronome also defragments on demand, but needs exact fragmentation statistics from the previous collection.

Conclusion

TODO: Mark-region is a new collector family; opportunistic defragmentation mixes copying and marking in one pass; immix achieves all three performance objectives.

References

- Stephen M. Blackburn, Kathryn S. McKinley. *Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance*. PLDI, 2008.